МИНОБРНАУКИ РОССИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
“ВОРОНЕЖСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ”
(ФГБОУ ВО «ВГУ»)

Факультет компьютерных наук

Кафедра программирования и информационных технологий

Курсовая работа

# Разработка React-приложения для отслеживания установки рекламных баннеров в городе

09.03.02 Информационные системы и технологии

Программная инженерия в информационных системах

Зав. кафедрой _____ С.Д. Махортов, к.ф.-м.н., доцент __.__.20__

Обучающийся _____ Д.В. Языкова, 3 курс, д/о

Руководитель _____ А.И. Чекмарев, ст. преподаватель

Воронеж 2025

# СОДЕРЖАНИЕ

ВВЕДЕНИЕ ..... 3
1 Аналитическая часть ..... 4
1.1 Обзор технологий: React и Leaflet ..... 4
1.2 Требования к приложению ..... 5
2 Проектирование приложения ..... 7
2.1 Архитектура приложения ..... 7
2.1.1 Структура данных ..... 7
2.1.2 Взаимодействие с пользователем ..... 8
2.2 Серверная часть и взаимодействие с базой данных ..... 8
2.3 Взаимодействие клиент-сервер ..... 9
3 Практическая часть ..... 10
3.1 Авторизация пользователей ..... 10
3.2 Сохранение и получение данных точек ..... 11
3.3 Загрузка и обработка CSV файлов ..... 11
3.4 Отображение карты и точек ..... 12
3.5 Режим добавления точек ..... 13
3.6 Управление статусами и датами установки ..... 14
3.7 Сохранение данных в CSV-файл ..... 15
ЗАКЛЮЧЕНИЕ ..... 16
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ ..... 18

# ВВЕДЕНИЕ

Актуальность темы разработки приложений для отслеживания установки рекламных баннеров в городе обусловлена необходимостью эффективного контроля и управления размещением наружной рекламы. В условиях современного городского пространства большое количество рекламных конструкций требует системного подхода к их учёту, своевременному обновлению данных и контролю статуса установки.

Использование современных веб-технологий, таких как React, позволяет создавать интерактивные и удобные в использовании интерфейсы, а библиотека Leaflet предоставляет мощные инструменты для визуализации географических данных на карте. Совмещение этих технологий даёт возможность разработать приложение, которое наглядно отображает состояние установки рекламных баннеров, облегчая работу специалистов и повышая прозрачность процессов.

Целью данной курсовой работы является разработка React-приложения с использованием библиотеки Leaflet для визуального отслеживания установки рекламных баннеров в городе. В рамках работы поставлены следующие задачи:

- анализ требований и технологий,
- проектирование архитектуры приложения,
- реализация функционала отображения и редактирования статусов баннеров,
- обеспечение удобного взаимодействия пользователя с картой и данными.

# 1 Аналитическая часть

## 1.1 Обзор технологий: React и Leaflet

В современных городах наружная реклама играет важную роль в информировании жителей и привлечении внимания к товарам и услугам. Эффективное управление установкой рекламных баннеров требует точного учёта их расположения и статусов установки. Традиционные методы ведения учёта зачастую связаны с использованием бумажных журналов или разрозненных электронных таблиц, что приводит к ошибкам, дублированию данных и снижению оперативности обновления информации.

Для повышения эффективности контроля и мониторинга установки рекламных конструкций целесообразно использовать интерактивные геоинформационные системы, позволяющие визуально отображать расположение баннеров на карте с возможностью изменения их статуса в режиме реального времени.

Для реализации поставленной задачи был выбран стек технологий, включающий React и библиотеку Leaflet.

React — популярная JavaScript-библиотека для создания пользовательских интерфейсов. React обеспечивает компонентный подход к разработке, что упрощает построение масштабируемых и поддерживаемых приложений. Использование хуков React (useState, useEffect, useCallback) позволяет эффективно управлять состоянием приложения и оптимизировать его работу [1].

Leaflet — легковесная и мощная библиотека для работы с интерактивными картами. Leaflet поддерживает различные типы картографических слоёв, работу с маркерами, всплывающими окнами и кластеризацией, что позволяет удобно визуализировать большое количество точек на карте [2].

react-leaflet — обёртка для интеграции Leaflet в React-приложения, обеспечивающая создание и управление компонентами карты.

react-leaflet-markercluster — расширение для кластеризации маркеров, позволяющее группировать близко расположенные точки, улучшая визуальное восприятие и производительность при большом количестве объектов.

PapaParse — библиотека для парсинга CSV-файлов, используемая для загрузки и обработки данных о точках установки баннеров из внешних файлов [3].

## 1.2 Требования к приложению

Основные требования к разработанному приложению включают:

- Отображение карты с возможностью выбора различных базовых слоёв (например, OpenStreetMap, спутниковая карта Esri, топографическая карта).
- Визуализация точек установки рекламных баннеров с цветовой индикацией статусов: "не установлены" (красный), "в процессе установки" (оранжевый), "установлены" (зелёный).
- Возможность загрузки данных из CSV-файлов с координатами и статусами баннеров.
- Режим добавления новых точек на карту по клику.
- Возможность изменения статуса баннера через всплывающее окно с выбором и, при необходимости, вводом даты установки.
- Кластеризация маркеров для удобства отображения большого количества точек.
- Сохранение обновлённых данных в CSV-файл для дальнейшего использования.
- Авторизация пользователей, обеспечивающая безопасный доступ к функционалу приложения и индивидуальное хранение данных.

- Сохранение и управление точками установки баннеров на сервере с использованием базы данных PostgreSQL.

Анализ предметной области и требований показал, что использование React совместно с Leaflet и сопутствующими библиотеками является оптимальным решением для создания интерактивного веб-приложения по отслеживанию установки рекламных баннеров. Такой подход обеспечивает удобство работы с геоданными и возможность масштабирования проекта. Внедрение серверной части с авторизацией и базой данных PostgreSQL позволяет обеспечить безопасность и устойчивость приложения при работе с пользовательскими данными.

# 2 Проектирование приложения

## 2.1 Архитектура приложения

Разрабатываемое приложение построено по компонентному принципу с использованием библиотеки React, что позволяет разделить интерфейс на независимые, переиспользуемые части. В основе архитектуры лежит одностраничное приложение, обеспечивающее динамическое взаимодействие с пользователем без перезагрузки страницы.

Основные компоненты приложения:

- MapComponent — главный компонент, отвечающий за отображение карты, маркеров и взаимодействие с ними.
- MarkerClusterGroup — компонент для кластеризации маркеров, обеспечивающий удобный просмотр большого числа точек.
- Всплывающие окна (popups) для редактирования статуса баннеров.
- Элементы управления для загрузки CSV-файлов и переключения режимов работы (просмотр/добавление точек).
- Компоненты авторизации и управления сессией пользователя, обеспечивающие вход и безопасное хранение информации о точках текущего пользователя.

### 2.1.1 Структура данных

Для хранения информации о рекламных баннерах используется массив объектов, каждый из которых содержит следующие поля:

- id — уникальный идентификатор точки.
- lat и lng — географические координаты точки.
- status — статус установки баннера (0 — не установлен, 1 — в процессе установки, 2 — установлен).

- date — дата установки.
- userId — идентификатор пользователя, которому принадлежит точка (для разграничения данных на сервере).

Данные загружаются из CSV-файла и могут быть изменены пользователем в интерфейсе. При изменении статуса или добавлении новой точки происходит обновление состояния приложения.

### 2.1.2 Взаимодействие с пользователем

Приложение предусматривает два основных режима взаимодействия:

- Режим просмотра — пользователь может просматривать расположение баннеров на карте, видеть их статусы и подробную информацию во всплывающих окнах.
- Режим добавления — при активации данного режима пользователь может добавить новую точку на карту кликом. После добавления точки открывается окно для выбора статуса и ввода комментария.

Для изменения статуса баннера используется выпадающий список с вариантами статусов и поле для ввода даты установки, если статус изменяется на «установлен». Это обеспечивает удобство и точность обновления данных.

## 2.2 Серверная часть и взаимодействие с базой данных

Для обеспечения сохранности данных и разграничения доступа реализована серверная часть на Node.js с использованием Express.js. Сервер обрабатывает запросы на авторизацию пользователей и CRUD-операции с точками баннеров.

Основные функции серверной части:

- Аутентификация пользователей с использованием JWT-токенов для безопасного доступа.

- Приём и сохранение данных точек в базе данных PostgreSQL, связанной с конкретным пользователем.
- Предоставление API для получения, добавления, обновления и удаления точек, доступных только авторизованным пользователям.

### 2.3 Взаимодействие клиент-сервер

Клиентская часть при загрузке и изменении данных обращается к серверу через REST API, передавая JWT-токен в заголовках запросов для аутентификации. Сервер проверяет права доступа, обрабатывает запросы и возвращает актуальные данные.

Таким образом, архитектура приложения сочетает удобный и отзывчивый пользовательский интерфейс с надёжной и защищённой серверной частью, что соответствует современным требованиям к веб-приложениям с геопривязкой и пользовательскими данными.

# 3 Практическая часть

Основная функциональность приложения заключается в загрузке точек из CSV-файлов, отображении их на карте с цветовой индикацией статусов, добавлении новых точек вручную и сохранении изменённых данных обратно в CSV.

## 3.1 Авторизация пользователей

Для контроля доступа к данным реализован механизм аутентификации на основе JSON Web Token (JWT) [4]. Серверная часть построена с использованием Node.js и Express.js, а данные пользователей хранятся в базе PostgreSQL.

Для входа пользователя (POST /login) сервер принимает логин пользователя и пароль, проверяет наличие пользователя в базе и корректность пароля (рис. 1).

При успешной аутентификации создаётся JWT, содержащий идентификатор и логин пользователя, с временем жизни 1 час. Токен возвращается клиенту для последующей аутентификации запросов.

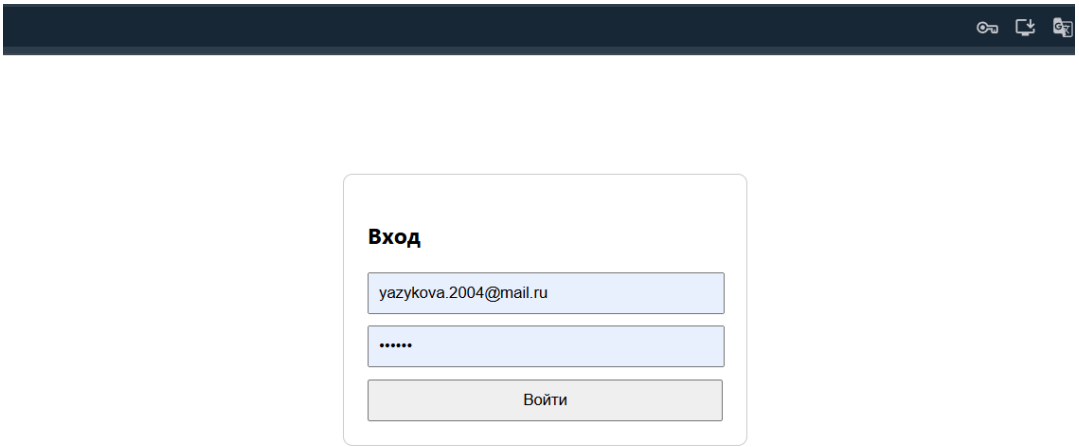

Рисунок 1 – Авторизация пользователя

### 3.2 Сохранение и получение данных точек

Для хранения информации о рекламных баннерах используется таблица points в базе данных PostgreSQL, где каждая запись связана с конкретным пользователем через user_id.

```
// Эндпоинт для сохранения точек
app.post( path: '/points', authenticateToken, async (req : Request<P, ResBody, ReqBody, ReqQuery, LocalsObj> , res : Respo

// Эндпоинт для получения точек текущего пользователя
app.get('/points', authenticateToken, async (req, res) : Promise<void> => {...});

// Эндпоинт для удаления точек
app.delete( path: '/points', authenticateToken, async (req : Request<P, ResBody, ReqBody, ReqQuery, LocalsObj> , res : Res
```

Рисунок 2 – Эндпоинты для работы с точками

Сохранение точек происходит с помощью POST-запроса(/points) — защищённый эндпоинт, принимающий массив точек в теле запроса (рис. 2). Для каждой точки проверяется корректность данных (наличие и тип координат), после чего точки последовательно сохраняются в базу в рамках транзакции. Это обеспечивает целостность данных — при ошибке все изменения откатываются.

Получение точек (GET /points): возвращает все точки, принадлежащие текущему аутентифицированному пользователю, сортированные по идентификатору (рис. 2).

Удаление точек (DELETE /points): удаляет все точки пользователя из базы, что позволяет, например, очистить данные перед загрузкой новой партии (рис. 2).

### 3.3 Загрузка и обработка CSV файлов

Пользователь может загрузить один или несколько CSV-файлов с точками (рис. 3). Для этого реализован обработчик handleFileUpload, который:

- Получает список файлов из input.
- С помощью Papa.parse парсит каждый файл с разделителем “;” и заголовком.
- Извлекает координаты (LAT, LNG), статус (STATUS) и дату установки (INSTALLATION_DATE).
- Фильтрует некорректные данные.
- Присваивает каждой точке уникальный ID.
- Обновляет состояние приложения, добавляя новые точки и их статусы.

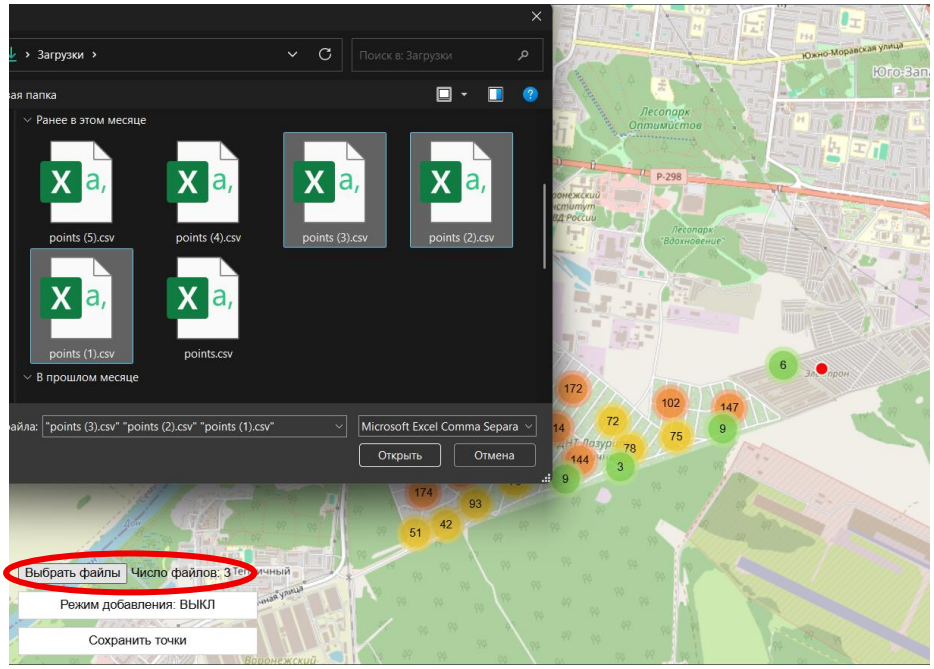


Рисунок 3 – Загрузка CSV-файла с точками

### 3.4 Отображение карты и точек

Карты отображаются с помощью компонента <MapContainer>. Пользователь может выбрать один из нескольких базовых слоёв карты в

правом верхнем углу страницы (OpenStreetMap, спутник Esri, топографическая карта), как показано на рисунке 4.

Для отображения точек используется кластеризация маркеров через MarkerClusterGroup, что позволяет удобно визуализировать большое количество точек.

Каждая точка отображается с цветной иконкой:

- Красный — "Не установлена"
- Оранжевый — "В процессе установки"
- Зелёный — "Установлена"

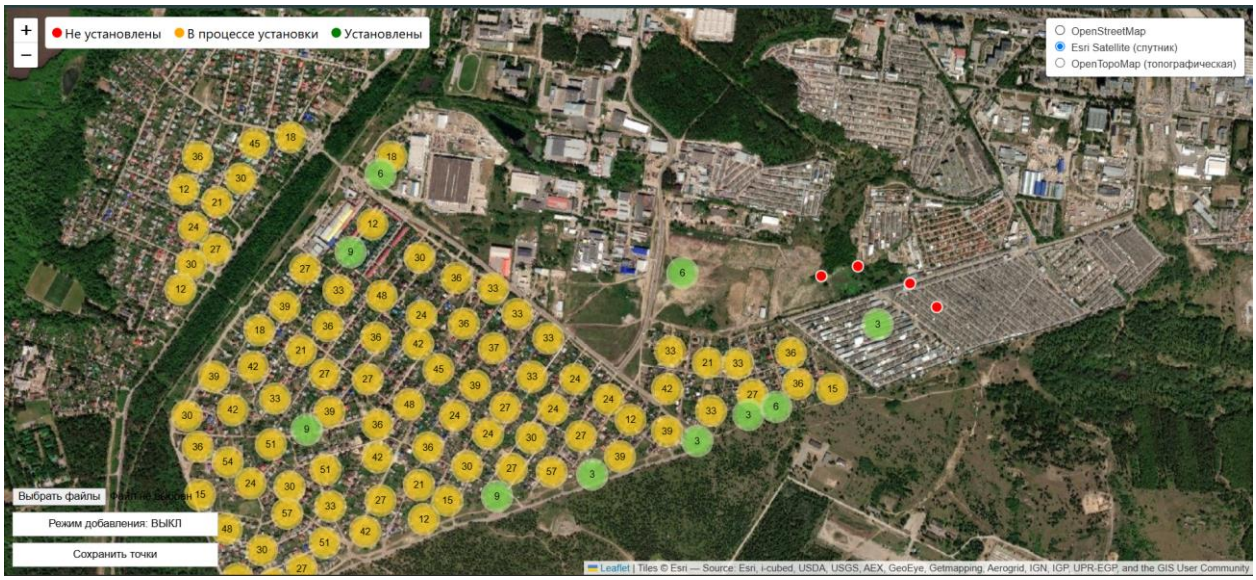


Рисунок 4 – Отображение карты и точек

При клике на маркер открывается всплывающее окно (Popup), где можно увидеть информацию о точке и изменить её статус.

### 3.5 Режим добавления точек

Режим добавления новых точек вручную — при включении режима (кнопка переключения) каждое нажатие левой кнопки мыши по карте добавляет новые точки с начальным статусом "Не установлена" (рис. 5). Это

реализовано через компонент AddPointOnClick, который использует хук useMapEvents для отслеживания кликов.

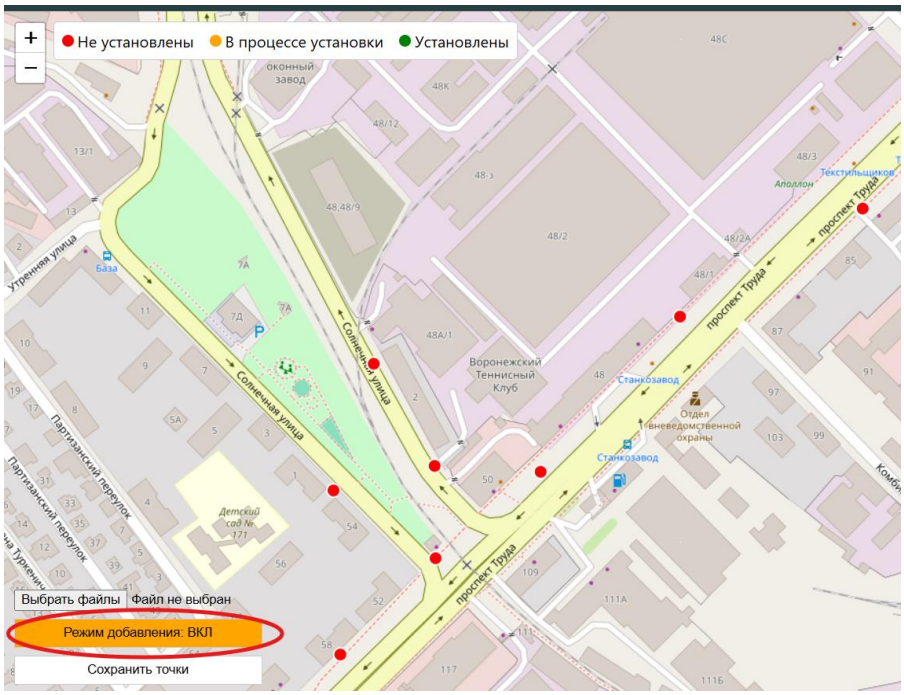


Рисунок 5 – Ручное добавление точек

### 3.6 Управление статусами и датами установки

Статус каждой точки можно изменить через выпадающий список в Popup (рис. 6). Если статус изменён на "В процессе установки", появляется поле для выбора даты установки.

Изменения статусов и дат синхронизируются с состоянием приложения, что позволяет корректно отображать и сохранять данные.

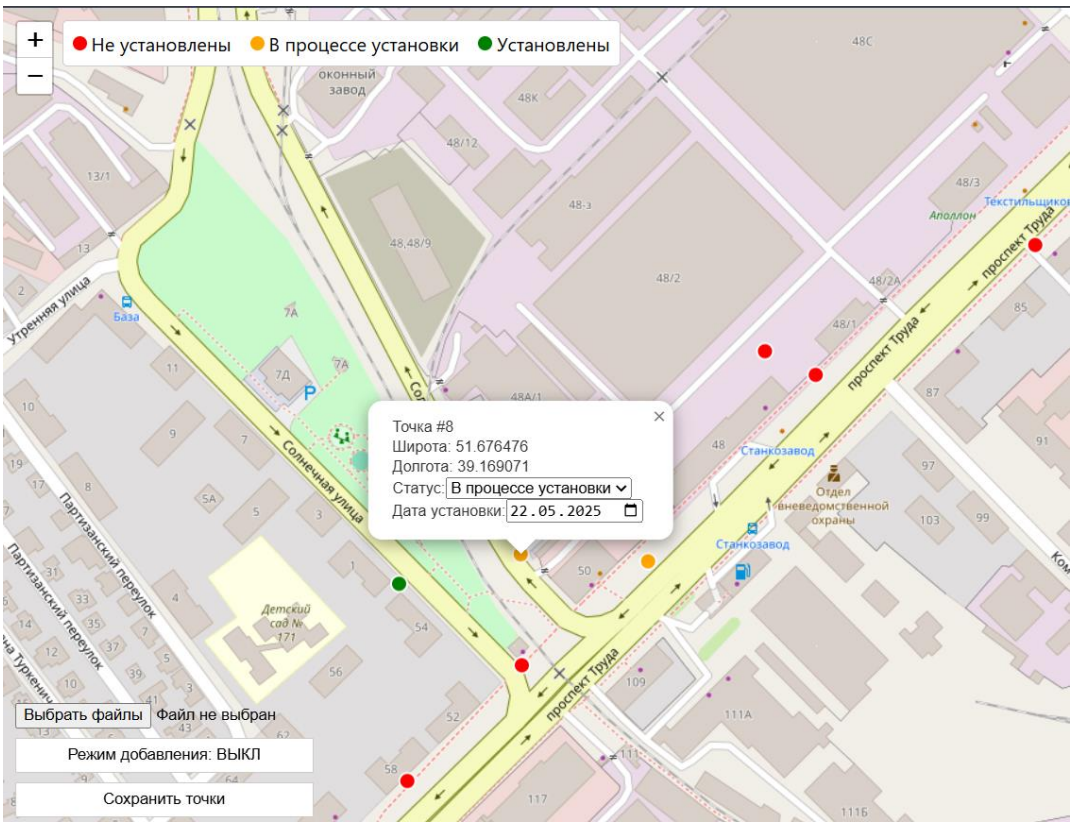


Рисунок 6 – Popup с данными о точке

### 3.7 Сохранение данных в CSV-файл

Пользователь может сохранить текущий список точек с их статусами и датами в CSV-файл (рис. 7).

Для реализации экспорта используется библиотека papaparse, которая поддерживает не только парсинг CSV, но и генерацию CSV-строк из JavaScript-объектов.

С помощью функции Papa.unparse из массива объектов формируется CSV-строка с заголовками. В настройках указывается разделитель “;” для совместимости с исходными файлами.

Для скачивания создаётся Blob[5] из CSV-строки с MIME-типом «text/csv;charset=utf-8;». Затем создаётся временный URL с помощью URL.createObjectURL.

Создаётся скрытая ссылка (<a>), у которой устанавливается атрибут href на созданный URL и download — имя файла (например, points.csv).

Программно вызывается клик по ссылке, инициируя скачивание. После этого временный URL освобождается вызовом URL.revokeObjectURL.

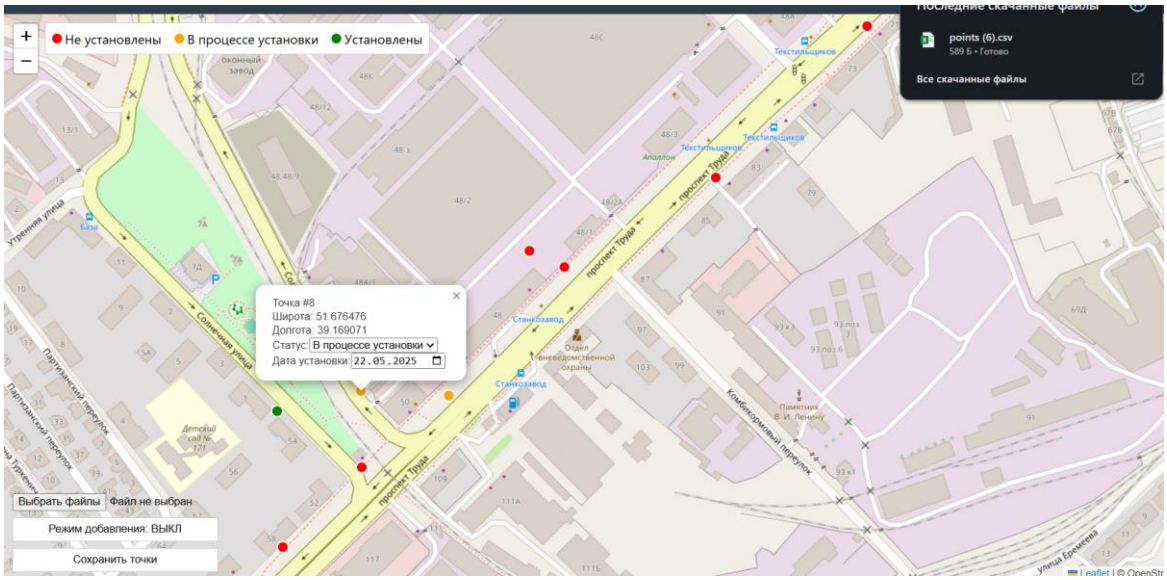


Рисунок 7 – Скачивание файла

| | A | B | C | D | E | F |
|---|---|---|---|---|---|---|
| 1 | ROWNUM | LAT | LNG | STATUS | INSTALLATION_DATE | |
| 2 | 1 | 51.677163 | 39.168405 | not_installed | | |
| 3 | 2 | 51.676432 | 39.170229 | in_progress | | |
| 4 | 3 | 51.675195 | 39.168041 | not_installed | | |
| 5 | 4 | 51.677483 | 39.171753 | not_installed | | |
| 6 | 5 | 51.678214 | 39.173748 | not_installed | | |
| 7 | 6 | 51.679451 | 39.175744 | not_installed | | |
| 8 | 7 | 51.676310 | 39.167965 | installed | | |
| 9 | 8 | 51.676476 | 39.169070 | in_progres | 22.05.2025 | |
| 10 | 9 | 51.675851 | 39.169081 | not_installed | | |
| 11 | 10 | 51.677620 | 39.171291 | not_installed | | |
| 12 | | | | | | |
| 13 | | | | | | |
| 14 | | | | | | |
| 15 | | | | | | |
| 16 | | | | | | |

Рисунок 8 – Отображение данных в CSV-файле

На рисунке 8 показано, как отображаются данные о каждой точке в CSV-файле.

# ЗАКЛЮЧЕНИЕ

В рамках данной курсовой работы было разработано React-приложение для отслеживания установки рекламных баннеров в городе. Приложение позволяет загружать данные о расположении баннеров из CSV-файлов, визуализировать их на интерактивной карте с помощью библиотеки Leaflet, а также добавлять новые точки, редактировать статус и дату установки. Реализована возможность сохранения обновлённых данных в CSV-файл, что обеспечивает удобство в работе с информацией и её дальнейшую обработку.

Для обеспечения безопасности и персонализации работы с данными реализована серверная часть с авторизацией пользователей на основе JSON Web Token (JWT). Пользователи проходят аутентификацию, после чего получают доступ к собственным данным, которые хранятся в базе данных PostgreSQL. Все точки рекламных баннеров связаны с конкретным пользователем, что позволяет надежно сохранять и управлять информацией в базе данных.

Разработанное приложение способствует автоматизации процесса мониторинга рекламных конструкций, упрощая контроль и анализ их установки в городской среде. Использование современных технологий React и Leaflet обеспечивает высокую производительность и удобство пользовательского интерфейса, а интеграция с серверной частью на Node.js и PostgreSQL — безопасность и надёжность хранения данных.

Исходный код проекта доступен в открытом репозитории на GitHub (https://github.com/KadZumi77/ReactLeaflet).

# СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. REACTJS.ORG: React – библиотека для создания пользовательских интерфейсов: [сайт]. – URL: https://reactjs.org/ (дата обращения: 1.05.2025). – Режим доступа: для всех пользователей. – Текст: электронный.

2. LEAFLETJS.COM: Leaflet – современная библиотека для интерактивных карт: [сайт]. – URL: https://leafletjs.com/ (дата обращения: 10.05.2025). – Режим доступа: для всех пользователей. – Текст: электронный.

3. PapaParse: Библиотека для работы с CSV в JavaScript: [сайт]. – URL: https://www.papaparse.com/ (дата обращения: 10.05.2025). – Режим доступа: для всех пользователей. – Текст: электронный.

4. HABR.COM: Аутентификация в React: [сайт]. – URL: https://habr.com/ru/companies/otus/articles/759222/ (дата обращения: 30.05.2025). – Режим доступа: для всех пользователей. – Текст: электронный.

5. MDN WEB DOCS: Работа с файлами и Blob в JavaScript: [сайт]. – URL: https://developer.mozilla.org/ru/docs/Web/API/Blob (дата обращения: 15.05.2025). – Режим доступа: для всех пользователей. – Текст: электронный.